

Agent context

LESSON 18 · MODULE 4 — TOOLS & WORKFLOWS

What you'll learn

- What a **context** is: named standing knowledge the agent carries on *every* call
- How to attach contexts at create time with `contexts=[...]`, and add or remove them **between calls**
- Why context lives in the **system prompt** — no retrieval step, unlike RAG
- The picking rule that separates **context vs. RAG vs. memory** so you stop confusing them

Key concepts

Context is named background the agent always carries. Company information, support hours, the current quarter, your brand voice — small, stable facts the agent should simply *know*. A context is a named string (`create_context(name, value)`) that lives in the system prompt, so the agent has it on every single call without looking anything up. No knowledge base, no documents, no retrieval.

TAKEAWAY

Context is named background the agent always carries.

Attach it at create time with `contexts=[...]`. Pass a list of context objects into `create_agent` and they're baked into the system prompt from the first run. Ask for the support hours and the answer comes straight from the context — there's no retrieval step to wait on, because the fact was already in front of the model.

Contexts are dynamic between calls — add and remove on the fly.

Say maintenance gets scheduled.

`agent.add_context(maintenance)` folds it in mid-session and the agent uses it on the next run — no recreate, same agent. When the window's over, `agent.remove_context(maintenance)` takes it back out. You're changing the agent's standing knowledge between calls, not rebuilding it.

TAKEAWAY

Contexts are dynamic between calls — add and remove on the fly.

Dynamic between calls is not per-message templating. This is the trap. There's no per-user variable substitution *inside* a single `agent.run` — context changes apply to subsequent calls, not to one message mid-flight. If you need per-user values, give each user their own context object, or just put the value in the message itself.

TAKEAWAY

Dynamic between calls is not per-message templating.

Context vs. RAG vs. memory — the three get confused, so here's the rule. **Context** is small, stable knowledge that's always in the system prompt and applies to every call — company facts, brand voice, a maintenance notice; the agent never goes looking for it. **RAG** is a document corpus the agent *retrieves* from per query; large and searchable, only the relevant chunks reach any one answer (lessons 10–13). **Memory** is the conversation itself — what the user said earlier in the session (lessons 13–14). If it's small, stable, and always-known →

TAKEAWAY

Context vs. RAG vs. memory — the three get confused, so here's the rule.

context. A big pile of documents you search → RAG. The back-and-forth of the conversation → memory. Most real agents use all three.

Code shown in this lesson

PYTHON

```
from lyzr import Studio

studio = Studio(api_key="...")

# Two contexts – each is a named string.
company = studio.create_context(
    name="company_information",
    value="Acme Corp – industrial IoT sensors. Founded 2019. Current quarter: Q2.",
)
support = studio.create_context(
    name="support_information",
    value="Support hours: 9am–6pm ET, Mon–Fri. Status page: status.acme.com.",
)
```

PYTHON

```
# Attach both at create time via the contexts parameter – no KB, no documents,
# no tools. Pure standing knowledge, baked into the system prompt.
agent = studio.create_agent(
    name="support", provider="gpt-4o",
    role="customer support agent",
    goal="answer customer questions",
    contexts=[company, support],
)

# Answered straight from the support context – hours and status page, no retrieval.
agent.run("What are your support hours?")
```

PYTHON

```
# The dynamic part – change standing knowledge between calls.
maintenance = studio.create_context(
    name="maintenance",
    value="Scheduled maintenance Sat 2am–4am ET; the dashboard may be briefly unavailable.",
)

agent.add_context(maintenance)      # folded in mid-session – same agent, no recreate
agent.run("Any planned downtime this weekend?")

agent.remove_context(maintenance)  # take it back out once maintenance is over
```

TRY THIS

- 1 Ask the agent a question answerable from a context (support hours), then a question that *isn't* covered. Notice it answers the first with no retrieval and is honest about the second — context is knowledge, not a search index.

- 2** Add the `maintenance` context, ask about downtime, then remove it and ask again. Confirm the agent's standing knowledge changed between calls without recreating the agent.
- 3** Try to make a context behave per-user by templating a name into one `run`. Watch it *not* work the way per-message substitution would — then fix it properly by giving the value in the message. This is the distinction worth feeling firsthand.